NEUF CHALLENGES · NEUF DÉCLICS

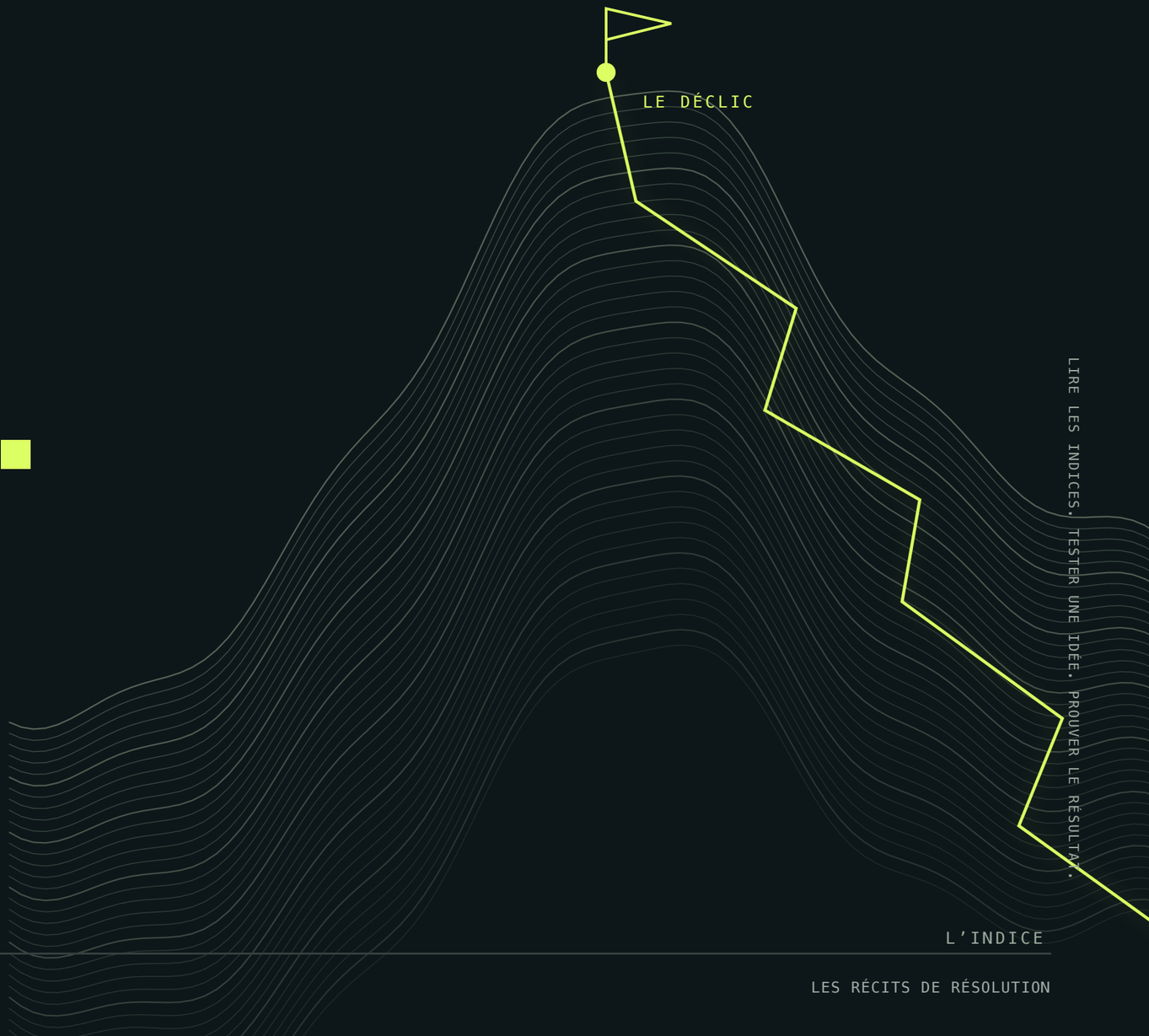
DERRIÈRE LES FLAGS.

Les agents ont trouvé les failles.
À nous de comprendre le chemin.

01

COMMENÇONS PAR UNE IDÉE SIMPLE

lost_in_embedding →



LE DÉFI · RETROUVER UN TEXTE CACHÉ

Le flag avait disparu.
Il restait des nombres.

FICHIER FOURNI

Dockerfile.json

```
model: "gpt2"
inputs_embeds:
  [ 0.1145, -0.3451, 0.2239, … ]
  [ … ]
  …
```

Extrait arrondi des premières valeurs.

24 groupes de nombres

768 nombres par groupe

Chaque groupe représente un morceau du texte, avec une information sur sa position.

L'IDÉE À RETENIR

Notre mission : **reconstituer le texte d'origine.**

COMPRENDRE · QU'EST-CE QU'UN EMBEDDING ?

Un morceau de texte.
Une ligne dans une table.

MORCEAU DE TEXTE

BA

Un « token »

→

TABLE PUBLIQUE DE GPT-2

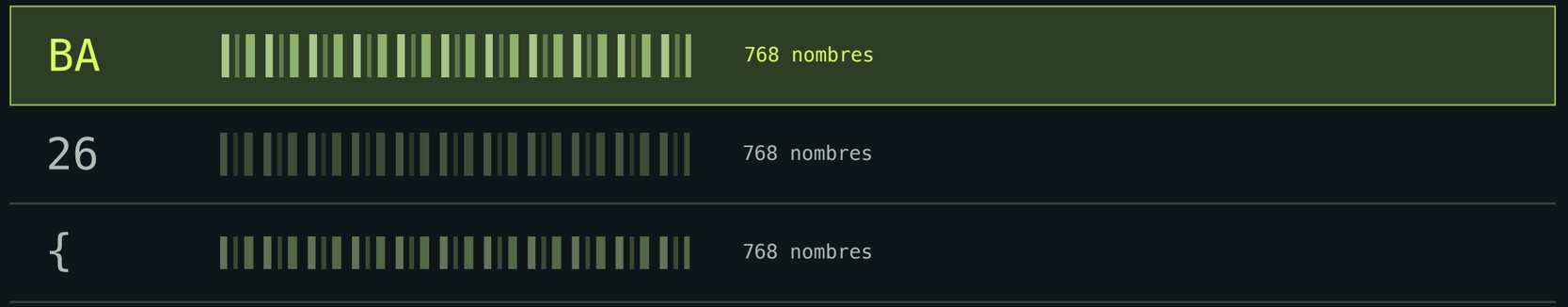


Schéma : les barres représentent des listes de nombres.

Dans ce fichier, on ajoute aussi : LA POSITION DU MORCEAU

Pour distinguer le premier, le deuxième, le troisième…

L'IDÉE À RETENIR

Le morceau de texte et sa position viennent de **deux tables connues.**

MINI-EXEMPLE · TROIS NOMBRES POUR COMPRENDRE

On sait ce qui a été ajouté.
On peut donc le retirer.

Un token	[2, 5, 1]	+	[1, 0, 3]	Sa position
Ce qu'on reçoit	[3, 5, 4]	−	[1, 0, 3]	Position connue
Ce qu'on retrouve	[2, 5, 1]	→	le token	Dans le dictionnaire

Exemple inventé à 3 dimensions. Dans le fichier réel, chaque ligne contient 768 nombres.

L'IDÉE À RETENIR

La soustraction s'effectue **nombre par nombre.**

LE DÉCLIC · ON CONNAÎT LES DEUX TABLES

Retirer la position.
Retrouver le morceau.

01

Lire un groupe

nombres du morceau

+ nombres de sa position

Les nombres reçus.

02

Retirer la position

nombres du morceau

~~nombres de sa position~~

Elle est dans la table publique.

03

Comparer à la table

BA ✓

Le morceau le plus proche est retrouvé.

L'IDÉE À RETENIR

Recommencer pour les **24 groupes**, puis assembler les morceaux.

VÉRIFIER · RETROUVER UN TEXTE NE SUFFIT PAS

Le texte était déjà lisible.
Les distances ont confirmé.

COMPARAISON DIRECTE

3,5 – 9,9

Distance aux lignes du dictionnaire.
Les bons tokens émergent déjà.

APRÈS RETRAIT DE LA POSITION

0,000 – 0,004

Des correspondances presque exactes.
La structure supposée est confirmée.

24 tokens → Le même flag dans les deux essais ✓

L'IDÉE À RETENIR

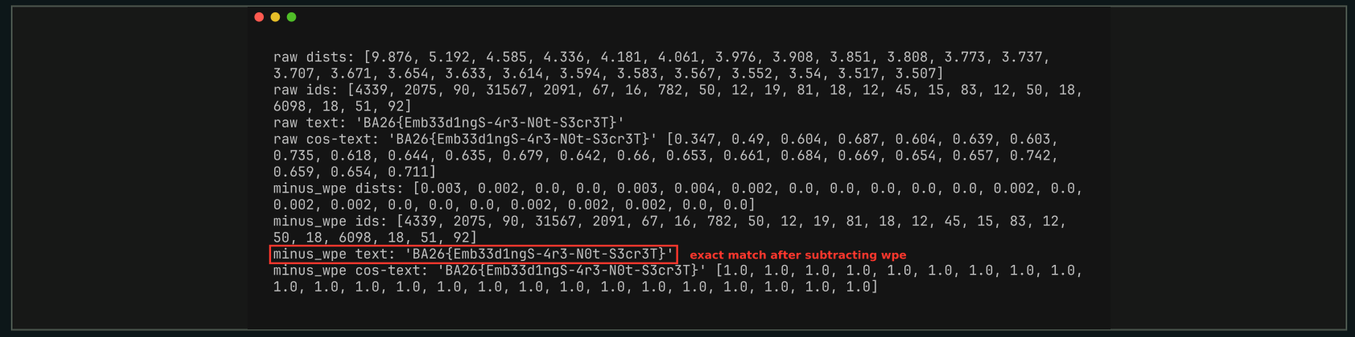
Une distance proche de zéro signifie : **« nous avons retrouvé la même ligne ».**

LA PREUVE · LES 24 MORCEAUX S’ASSEMBLENT

Le flag avait mis un camouflage. Le dictionnaire l’a reconnu.

BA	26	{	Emb	33	d	1	ng	S	…	}

BA26{Emb33d1ngS-4r3-N0t-S3cr3T}



```
raw dists: [9.876, 5.192, 4.585, 4.336, 4.181, 4.061, 3.976, 3.908, 3.851, 3.808, 3.773, 3.737,
3.707, 3.671, 3.654, 3.633, 3.614, 3.594, 3.583, 3.567, 3.552, 3.54, 3.517, 3.507]
raw ids: [4339, 2075, 90, 31567, 2091, 67, 16, 782, 50, 12, 19, 81, 18, 12, 45, 15, 83, 12, 50, 18,
6098, 18, 51, 92]
raw text: 'BA26{Emb33d1ngS-4r3-N0t-S3cr3T}'
raw cos-text: 'BA26{Emb33d1ngS-4r3-N0t-S3cr3T}' [0.347, 0.49, 0.604, 0.687, 0.604, 0.639, 0.603,
0.735, 0.618, 0.644, 0.635, 0.679, 0.642, 0.66, 0.653, 0.661, 0.684, 0.669, 0.654, 0.657, 0.742,
0.659, 0.654, 0.711]
minus_wpe dists: [0.003, 0.002, 0.0, 0.0, 0.003, 0.004, 0.002, 0.0, 0.0, 0.0, 0.0, 0.0, 0.002, 0.0,
0.002, 0.002, 0.0, 0.0, 0.0, 0.002, 0.002, 0.002, 0.0, 0.0]
minus_wpe ids: [4339, 2075, 90, 31567, 2091, 67, 16, 782, 50, 12, 19, 81, 18, 12, 45, 15, 83, 12,
50, 18, 6098, 18, 51, 92]
minus_wpe text: 'BA26{Emb33d1ngS-4r3-N0t-S3cr3T}'   exact match after subtracting wpe
minus_wpe cos-text: 'BA26{Emb33d1ngS-4r3-N0t-S3cr3T}' [1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0,
1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0]
```

PREUVE CONSERVÉE · CLIQUER POUR AGRANDIR

24 / 24

Morceaux retrouvés.
Correspondances presque exactes
après retrait de la position.

Solve : Claude

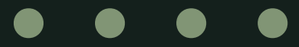
L’IDÉE À RETENIR

Dans ce cas, transformer le texte en nombres **ne protégeait pas son contenu.**

LE DÉFI · ACCÉDER AU COMPTE ADMINISTRATEUR

Un compte admin. Deux portes d’entrée.

CONNEXION CLASSIQUE



Email + mot de passe

Le mot de passe de l’admin reste inconnu.

CONNEXION GOOGLE

ATTESTATION D’IDENTITÉ

Numéro du compte

« Voici qui se connecte. »

Une attestation transmise au site

Le site lit le numéro du compte.

LA FAILLE

Sur le chemin « legacy », **le site ne vérifiait pas la signature.**

LES IMPASSES · CHANGER LE RÔLE NE SUFFIT PAS

Écrire « admin »…
La promotion a été refusée.

QUESTION TESTÉE	CE QUE LE SERVICE A RÉPONDU
01 Ajouter « rôle : admin » Peut-on demander soi-même les privilèges ?	× Le compte reste utilisateur.
02 Garder l’email, changer le numéro Le site reconnaît-il un compte par son email ?	× Un autre compte est créé.
03 Garder le même numéro « sub » Est-ce lui qui détermine le compte ?	✓ Le même compte est retrouvé.

`sub` Le numéro du compte Google.
Le rôle est ensuite lu dans le compte enregistré.

L’IDÉE À RETENIR

Il fallait retrouver **l’identifiant du vrai compte admin.**

LE DÉCLIC · L’INDICE ÉTAIT AU PIED DE LA PAGE

Le numéro était trouvable.
Le site le traitait comme un secret.

01

Le pied de page

Tech Corp Chandler

Une adresse Gmail de contact

PIED DE PAGE · INDICE REPRÉSENTÉ

Un indice cité dans le journal, au pied de la page d’accueil.

02

La recherche du compte

Google People API

Avec une session Google de test

L’adresse permet ici de retrouver un identifiant.

03

Le bon numéro

`1034…4574`

GAIA ID / sub · abrégé

À placer dans l’attestation fabriquée.

L’IDÉE À RETENIR

Le déclic : **chercher l’identité** au lieu de deviner un mot de passe.

LE RÉSULTAT · SOLVE CONSIGNÉ À 21:25

Le bon identifiant.
Une session administrateur.

Attestation fabriquée → Compte existant retrouvé → ADMIN
RÔLE DU COMPTE RETROUVÉ

EXTRAIT DU WRITE-UP CONSERVÉ · TRANSCRIPTION

`socialLogin`	**memberId: 1 · role: admin**
`/api/flag`	**flag retourné**

La route « /api/admin/flag » était un leurre : même connecté, il renvoyait 404.

`BA26{sub_is_not_a_secret_but_treating_it_like_a_password_is}`

L'IDÉE À RETENIR

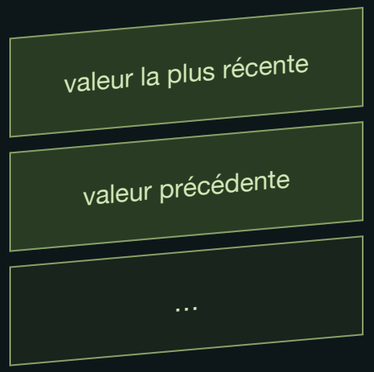
Vérifier **la preuve d'identité**, pas seulement lire le numéro.

LE DÉFI · COMPRENDRE UNE MACHINE INCONNUE

La machine parlait couramment. On cherchait le dictionnaire.

DES CODES, MAIS QUELLES ACTIONS ?

```
03  →  ?
04  →  ?
19  →  ?
```



Une « machine à pile »

Elle range des valeurs en pile, puis les copie, les déplace ou fait des calculs.

Le coup de pouce : 4 programmes d'exemple + leurs résultats attendus

L'IDÉE À RETENIR

Les exemples vont permettre de **retrouver le sens des codes.**

COMPRENDRE · UN MINI-EXEMPLE POUR VOIR LE PRINCIPE

Copier ou échanger ?
Le résultat les départage.

EXEMPLE PÉDAGOGIQUE INVENTÉ · LES VRAIS TESTS VIENNENT ENSUITE

PILE DE DÉPART

3	sommet
7	

On traite d'abord la valeur du dessus.

Instruction inconnue
puis afficher le sommet

HYPOTHÈSE A

Copier le sommet

3	copie
3	
7	

→ SORTIE 3

✓ compatible

HYPOTHÈSE B

Échanger les deux

7	
3	

→ SORTIE 7

× éliminée

SI LA SORTIE ATTENDUE EST 3 « Échanger » ne peut pas être la bonne instruction.

L'IDÉE À RETENIR

Un exemple connu sert à **rejeter une interprétation qui ne colle pas.**

LE SOLVE · APPLIQUER CE TEST À TOUS LES DICTIONNAIRES

Les exemples font le tri.
Un seul dictionnaire reste.

01

D’abord, les actions sur la pile

Essayer les associations entre codes et actions.

Exemple 1 | Exemple 2

1 correspondance

02

Puis, les opérations de calcul

Garder les associations qui produisent les bonnes sorties.

Exemple 3 | Exemple 4

1 correspondance

346 possibilités après l’exemple 3 → **1** après l’exemple 4

Dans l’interprétation correcte des nombres signés.

L’IDÉE À RETENIR

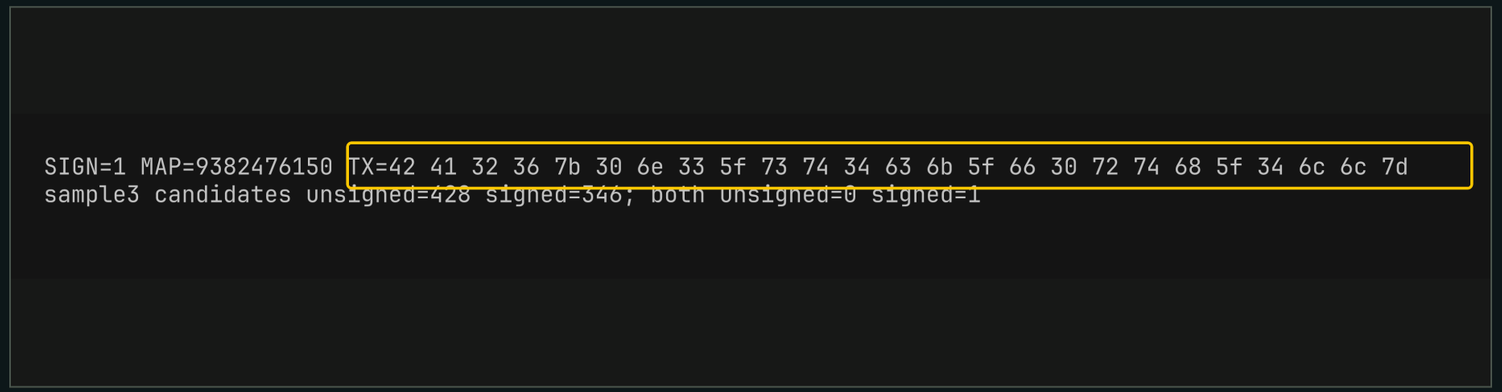
La transmission inconnue est lue **avec ce dictionnaire confirmé.**

LA PREUVE · LE DICTIONNAIRE RETROUVÉ FONCTIONNE

La machine retrouve sa langue. Le message devient lisible.

03	copier le sommet
04	émettre un octet
19	additionner

Trois exemples du dictionnaire retrouvé.
Codes écrits en hexadécimal.



PREUVE CONSERVÉE · CLIQUER POUR AGRANDIR

Solve : **Codex** · Sortie enregistrée du solveur.

`BA26{0n3_st4ck_f0rth_4ll}`

L'IDÉE À RETENIR

Les résultats connus ont servi de **mode d'emploi pour lire l'inconnu.**

LE DÉFI · OUVRIR UN ENREGISTREMENT PROTÉGÉ

Un flag en clair ?
C’était trop beau.

FICHIER FOURNI

`beacon_flash.bin`



Une copie de la mémoire
d’un appareil ESP32-C6.

→

CE QUE LE PROGRAMME VÉRIFIE

Code d’activation ?

_ _ _ _ _ _ _ _

Le bon code autorise
le décodage de l’enregistrement.

LE PIÈGE ~~`BA26{n0t_th15_0n3_k33p_d1gg1ng}`~~ Présent en clair, mais ce n’est pas le résultat recherché.

L’IDÉE À RETENIR

Trouver un texte qui ressemble à un flag **ne suffit pas à prouver le solve.**

LE DÉCLIC · REFAIRE LE CONTRÔLE D’ACTIVATION

La balise calcule le code.
L’agent refait le calcul.

DEUX TEXTES LISIBLES DANS LE DUMP

`LilyGO T7-C6`

`BA26-BEACON-01`

→

LE PROGRAMME EN FAIT DEUX NOMBRES

`56ea3ff6`

`cd1014a9`

Des « empreintes » calculées.

→

COMBINÉS COMME DANS LE PROGRAMME

`B80FEF82`

Le code attendu.

CODE FOURNI `?` = ? CODE ATTENDU `B80FEF82` →

Si égal : autoriser le décodage.

L’IDÉE À RETENIR

Le calcul et ses ingrédients sont présents : **la réponse peut être reconstruite.**

COMPRENDRE · LE CONTRÔLE N’EST PAS LE DÉCODAGE

Le code ouvre le passage.
Un autre calcul lit le contenu.

LES DEUX EMPREINTES

`56ea3ff6`

`cd1014a9`

Mêmes ingrédients,
deux usages.

01 **Vérifier l’accès**

Calculer le code attendu.

`B80FEF82`

02 **Décoder le contenu**

Produire une suite de nombres.

• • • …

31 OCTETS DU DUMP

Données chiffrées

avec

SUITE CALCULÉE

Un nombre par octet

→

COMBINAISON BIT À BIT

`BA26{…}`

L’IDÉE À RETENIR

Le solveur reproduit **le contrôle, puis le décodage de 31 octets.**

LA PREUVE · L'ENREGISTREMENT DÉCODÉ

Suivre la bonne branche. Lire le vrai enregistrement.

CODE RETROUVÉ

B80FEF82 ✓

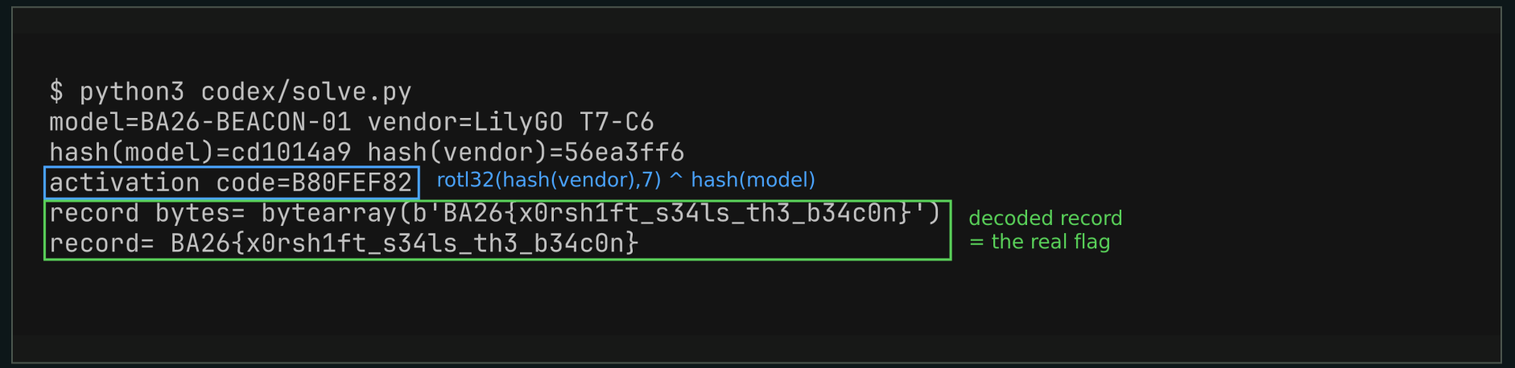
Calcul du code

↓

Décodage des données

↓

Enregistrement lisible



PREUVE CONSERVÉE · CLIQUER POUR AGRANDIR

Solve : **Codex** · Résultat du solveur hors ligne.

`BA26{x0rsh1ft_s34ls_th3_b34c0n}`

L'IDÉE À RETENIR

La preuve vient du **chemin de décodage**, pas d'un texte trouvé au hasard.

LE DÉFI · UNE APPLICATION DE FICHIERS

Le serveur rangeait les fichiers.
On a ouvert les autres tiroirs.

REQUÊTE CONSIGNÉE DANS LE JOURNAL	`GET /raw?filename=../../challenge.db`

DOSSIER ATTENDU	`/app/uploads/<compte>/`	Les fichiers de notre compte.
CHEMIN CONSTRUIT	`/app/~~uploads/<compte>/~~../../challenge.db`	Chaque `../` remonte d'un dossier.
FICHIER RÉELLEMENT VISÉ	`/app/challenge.db`	La base de comptes, hors de notre dossier.

`/download/…` bloque cette sortie `/raw?filename=…` la permet

Reconstitution pédagogique. Aucune capture HTTP conservée.

L'IDÉE À RETENIR

Assembler un chemin **ne garantit pas qu'il reste dans le dossier autorisé.**

LE DÉCLIC · NE PAS S’ARRÊTER AU PREMIER TEXTE PLAUSIBLE

Un faux flag dans la base.
Une vraie piste juste à côté.

LE PIÈGE Un mot de passe ressemblait à un flag. LEURRE

01

Lire les comptes

La base contient aussi un compte administrateur.

→

02

Se connecter

Le tableau de bord admin montre d’autres fichiers.

→

03

Suivre l’indice

`flag.txt` indique que le flag est caché dans `flag.maze`.

L’IDÉE À RETENIR

Le bon chemin mène à **un labyrinthe de caractères.**

LE SOLVEUR · RECHERCHE EN LARGEUR, OU BFS

Explorer les voisins. Garder le chemin du retour.

LOGIQUE DU SOLVEUR CONSERVÉ · PSEUDOCODE

```
file ← [entrée]
tant que la file contient une case :
  prendre la prochaine case
  ajouter ses voisins libres non visités
  mémoriser leur case précédente
```

`#` = mur · `>` = porte · les lettres sont dans les couloirs

01 **Explorer par couches**

D’abord les voisins, puis leurs voisins.

02 **Retrouver la sortie**

Chaque case mémorise d’où elle vient.

03 **Remonter les prédécesseurs**

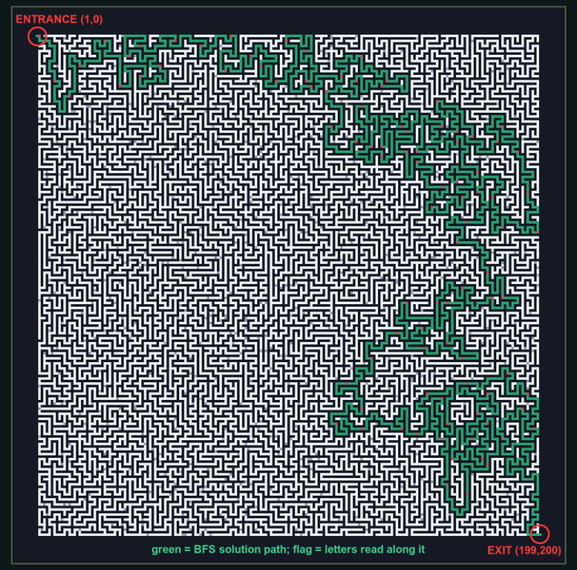
On garde uniquement le chemin entrée → sortie.

L’IDÉE À RETENIR

Lire **les lettres de ce chemin**, pas toutes les lettres du labyrinthe.

LA RÉSOLUTION · SUIVRE LE LABYRINTHE

Le chemin vers la sortie écrit le flag.



PREUVE CONSERVÉE · CLIQUER POUR AGRANDIR

201 × 201 cases dans le labyrinthe

1. Partir de l’entrée.
2. Explorer les passages accessibles.
3. Lire les lettres du chemin vers la sortie.

2 783 POSITIONS SUR LE CHEMIN

`BA26{tHis_Is_a_rEally_loNG_flag_…}`

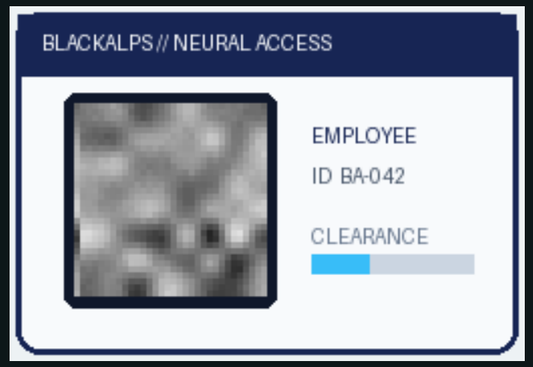
L’IDÉE À RETENIR

L’accès aux fichiers ouvre la porte. **Le parcours du labyrinthe donne la réponse.**

LE DÉFI · UNE IMAGE DÉCIDE DES DROITS

Pour entrer comme admin, il faut convaincre un modèle.

BADGE FOURNI



Classé « EMPLOYEE ».

→

LE SCANNER RENVOIE DES SCORES

EMPLOYEE	
INVALID	
ADMIN	≥ 99,5 %

Le seuil à dépasser pour obtenir l’accès.

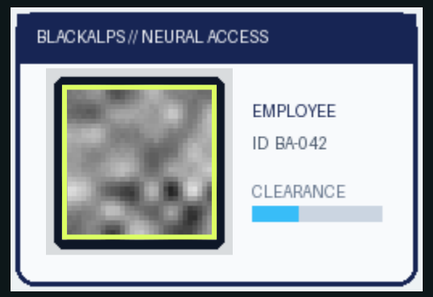
L’IDÉE À RETENIR

On peut modifier l’image et observer **le score renvoyé à chaque essai.**

L'INDICE · LE PRÉTRAITEMENT EST FOURNI, LE MODÈLE RESTE CACHÉ

Le modèle ne voit qu'un carré.
Nous contrôlons toutes ses cases.

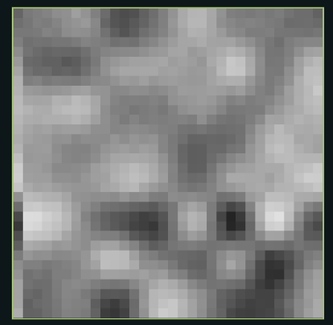
IMAGE FOURNIE · 256 × 176



Le scanner découpe
la zone encadrée.

→

CARRÉ · 96 × 96



Niveaux de gris,
puis moyenne par bloc de 3 × 3.

→

ENTRÉE DU MODÈLE · 32 × 32



Un bloc de neuf pixels identiques
fixe la valeur d'une case.

L'IDÉE À RETENIR

Modifier le bon carré suffit : **le reste du badge n'est pas vu par le modèle.**

LE DÉCLIC · UTILISER LE RETOUR DU MODÈLE

Tester des variations par paires. Suivre le meilleur signal.

POINT DE DÉPART Le badge employé. Les images aléatoires étaient surtout classées « INVALID ».

01

Varier dans les deux sens

```
badge + variation
badge − variation
```

20 directions essayées par paires :
40 images dans un lot.

→

02

Comparer les retours

MOINS ADMIN | PLUS ADMIN

Les différences de scores
indiquent les directions utiles.

→

03

Mettre le badge à jour



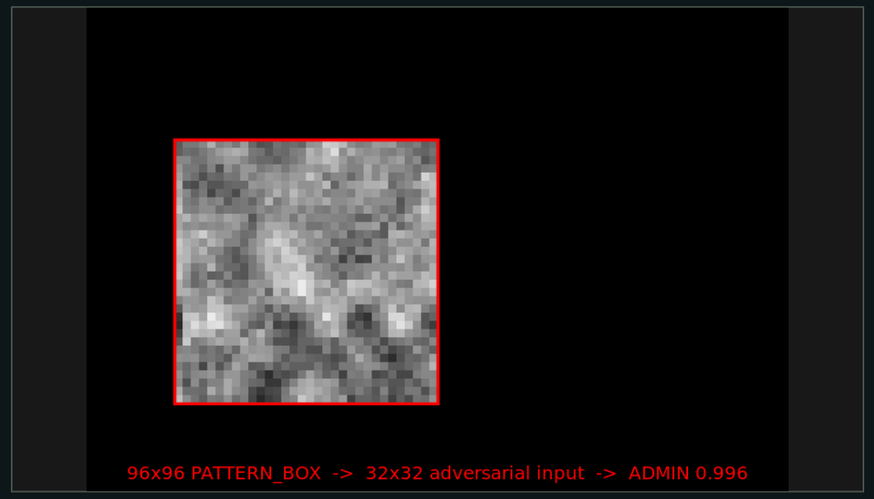
Combiner ces directions,
faire un pas, puis recommencer.

L'IDÉE À RETENIR

L'agent apprend **comment changer l'image** en observant la réponse du modèle.

LE RÉSULTAT · LE MODÈLE ACCORDE L’ACCÈS

Le badge ressemblait à du bruit.
Le modèle a reconnu le patron.



PREUVE CONSERVÉE · CLIQUER POUR AGRANDIR

SCORE ADMIN RAPPORTÉ DANS LE JOURNAL

99,6 %

Seuil demandé : 99,5 %.

ACCÈS ACCORDÉ

`BA26{r3ached_1h3_t0P}`

L’IDÉE À RETENIR

Une décision d’accès fondée sur une image peut être **pilotée par les scores exposés.**

LE DÉFI · RETROUVER LE SECRET TRANSMIS PAR RADIO

La balance était connectée. Et un peu trop bavarde.

ENREGISTREMENT RADIO

10 secondes de signal

Les échanges entre balance et passerelle.

FIRMWARE ESP32-C6

Le programme de la balance

Il révèle le protocole et les règles.

COUCHE 1 · RADIO

Signal → paquets — Retrouver les octets transmis.

COUCHE 2 · RÉSEAU ZIGBEE

Paquets → rapports — Retrouver la clé réseau dans l'appairage.

COUCHE 3 · APPLICATION

Fragments → flag — Reconstruire la clé liée à cette balance.

L'IDÉE À RETENIR

Trois étapes distinctes : **décoder, déchiffrer le réseau, ouvrir les fragments.**

ÉTAPE 1 · DÉCODER LE SIGNAL

Avant de déchiffrer, il faut retrouver les paquets.

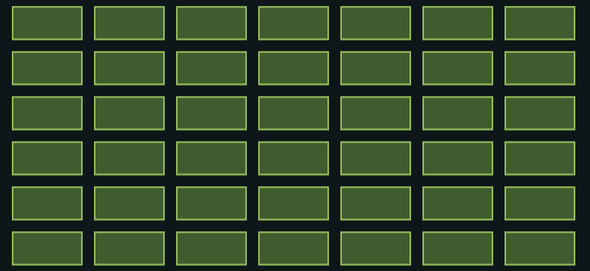
SIGNAL RADIO · SCHÉMA



Reconnaître les motifs utilisés par Zigbee.

→

PAQUETS RETROUVÉS



42 / 42

Contrôles d’erreur valides.

L’IDÉE À RETENIR

Le contrôle d’erreur confirme **que les octets ont été correctement décodés.**

ÉTAPE 2 · ÉCOUTER L’ENTRÉE DANS LE RÉSEAU

Au moment de rejoindre le réseau, la balance reçoit une clé.

L’ÉCHANGE D’APPAIRAGE EST DANS LA CAPTURE

La passerelle transmet une clé réseau, protégée à partir d’une clé par défaut connue.

CLÉ PAR DÉFAUT

`ZigBeeAlliance09`

→

RÈGLE ZIGBEE

Dériver une clé de transport

→

MESSAGE CAPTURÉ

Ouvrir « Transport-Key »

RÉSULTAT VÉRIFIÉ

Clé réseau retrouvée ✓

Le contrôle d’authenticité du message est valide.

→

RAPPORTS DÉCHIFFRÉS

6 fragments · 46 octets

Encore chiffrés par l’application.

L’IDÉE À RETENIR

La clé réseau devient accessible par son échange de distribution enregistré.

ÉTAPE 3 · RELIER LE MESSAGE À CETTE BALANCE

La dernière clé dépend de l'identité de la balance.

IDENTIFIANT RADIO · EUI-64 OBSERVÉ

`40:4c:ca:ff:fe:46:87:c4`

+

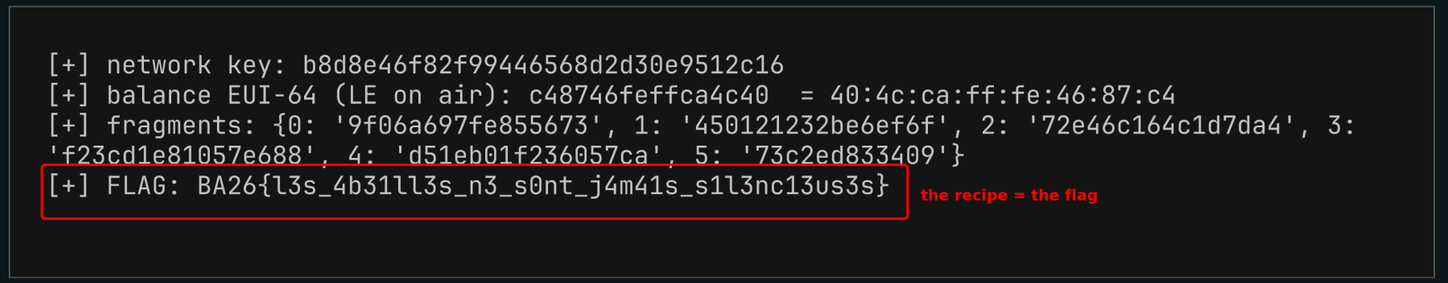
CONSTANTE DU FIRMWARE

« propolis »

+

NUMÉRO DU FRAGMENT

0, 1, 2, 3, 4 ou 5



PREUVE CONSERVÉE · CLIQUER POUR AGRANDIR

Reconstruire la clé de chaque fragment.

Appliquer RC4, puis assembler dans l'ordre.

L'identité personnalise la clé ; elle est observable dans la capture.

`BA26{l3s_4b31ll3s_n3_s0nt_j4m41s_s1l3nc13us3s}`

L'IDÉE À RETENIR

Le solve **reconstruit les clés** à partir de données connues, puis déchiffre avec RC4.

LE DÉFI · FAIRE ACCEPTER LE MÊME ORDRE AUX 22 PILOTES

Le message est simple.
Le faire accepter l'est moins.

ORDRE CIBLE **“Box Box Box!”**

Un appel au stand.

POUR FABRIQUER LA PREUVE

Clé privée

Secrète. Elle permet de signer.

→

SIGNATURE

Liée à ce message

→

POUR CONTRÔLER LA PREUVE

Clé publique

Connue. Elle permet de vérifier.

L'IDÉE À RETENIR

Les clés publiques sont dans le binaire. **Les clés privées nous manquent.**

driver_radio/JOURNAL.md : entrées 20:00 et 20:05 ; claude/send_exploit.py.

L'INDICE · LE NOMBRE TEMPORAIRE UTILISÉ PAR UNE SIGNATURE

Le hasard avait 255 possibilités.

255

Le « nonce »

Un nombre secret temporaire, utilisé pour une signature.

> À distinguer de la **clé privée**, le secret du pilote que l'on cherche.

LES ENTIERS DE 1 À 255 · TOUTE LA RECHERCHE

001	002	003	004	005	006	007	008	009	010	011	012	013	014	015	016	017
018	019	020	021	022	023	024	025	026	027	028	029	030	031	032	033	034
035	036	037	038	039	040	041	042	043	044	045	046	047	048	049	050	051
052	053	054	055	056	057	058	059	060	061	062	063	064	065	066	067	068
069	070	071	072	073	074	075	076	077	078	079	080	081	082	083	084	085
086	087	088	089	090	091	092	093	094	095	096	097	098	099	100	101	102
103	104	105	106	107	108	109	110	111	112	113	114	115	116	117	118	119
120	121	122	123	124	125	126	127	128	129	130	131	132	133	134	135	136
137	138	139	140	141	142	143	144	145	146	147	148	149	150	151	152	153
154	155	156	157	158	159	160	161	162	163	164	165	166	167	168	169	170
171	172	173	174	175	176	177	178	179	180	181	182	183	184	185	186	187
188	189	190	191	192	193	194	195	196	197	198	199	200	201	202	203	204
205	206	207	208	209	210	211	212	213	214	215	216	217	218	219	220	221
222	223	224	225	226	227	228	229	230	231	232	233	234	235	236	237	238
239	240	241	242	243	244	245	246	247	248	249	250	251	252	253	254	255

OBSERVER	La signature fournit une valeur liée au nonce.
COMPARER	Essayer les 255 possibilités et retrouver celle qui correspond.

L'IDÉE À RETENIR

L'agent retrouve le nonce. **Il lui manque encore le message signé.**

LE DÉCLIC · LE MESSAGE EST CACHÉ, SES FORMES SONT CONNUES

La radio variait ses messages. Pas tant que ça.

PREMIÈRE PISTE



Reproduire le hasard à partir de l'heure.

~~Piste finalement abandonnée.~~

LA PISTE RETENUE · MODÈLES DU SOLVEUR

MODE	`[pilote] Activate ECU mode {x}`
PLAN	`[pilote] Go plan {lettre}{signe}{tour}`
STAND	`[pilote] Pit window lap {tour}, {pneu}`

Quelques formats. De petites plages de valeurs.

≈ 800

MESSAGES POSSIBLES PAR PILOTE

Les essayer chacun, au lieu de déchiffrer l'échange.

L'IDÉE À RETENIR

Le solve final évite **la prédiction de l'heure et la synchronisation du hasard.**

LA MÉTHODE · UNE HYPOTHÈSE DOIT PASSER UN TEST

Une clé candidate.
Puis une vérification.

01
Signature observée
Fournie par Spy.

+

02
Nonce retrouvé
Parmi 255 valeurs.

+

03
Message possible
Une hypothèse parmi les modèles.

CES INFORMATIONS PERMETTENT DE CALCULER

Une clé privée candidate

↓

LA CLÉ PUBLIQUE CORRESPOND-ELLE AU PILOTE ?

NON Essayer un autre message. | OUI La clé est confirmée.

22 / 22 Clés privées confirmées.
Chaque candidate est contrôlée.

L'IDÉE À RETENIR

La clé publique connue sert de **test de confirmation.**

LE RÉSULTAT · SIGNER LE MESSAGE CIBLE AVEC CHAQUE CLÉ

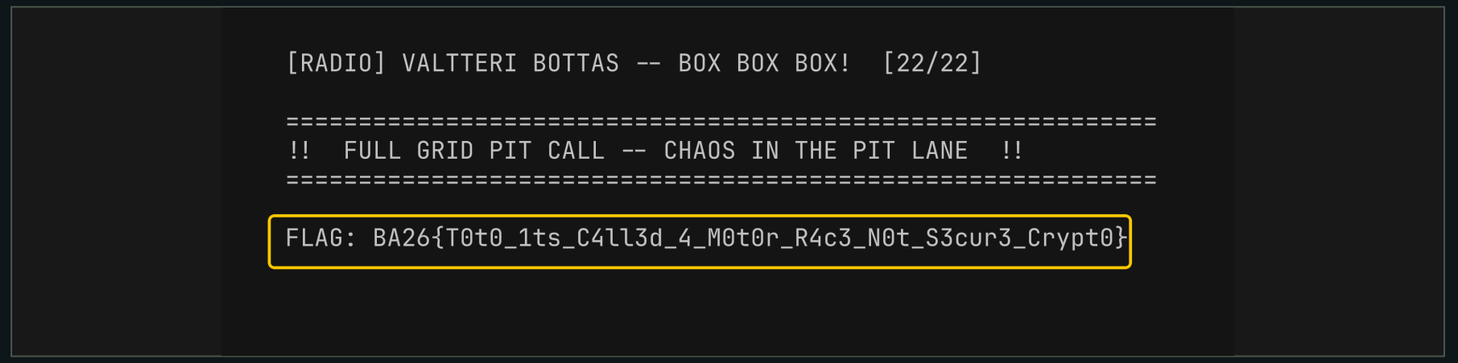
22 pilotes au stand. Adieu, pause café.

22/22

Commandes acceptées.

01	02	03	04	05	06	07	08	09	10	11
12	13	14	15	16	17	18	19	20	21	22

RÉSULTAT CONSIGNÉ · PAS UNE ATTAQUE EN DIRECT



PREUVE CONSERVÉE · CLIQUER POUR AGRANDIR

Le journal et `final_resp.txt` conservent les acceptations et le flag.

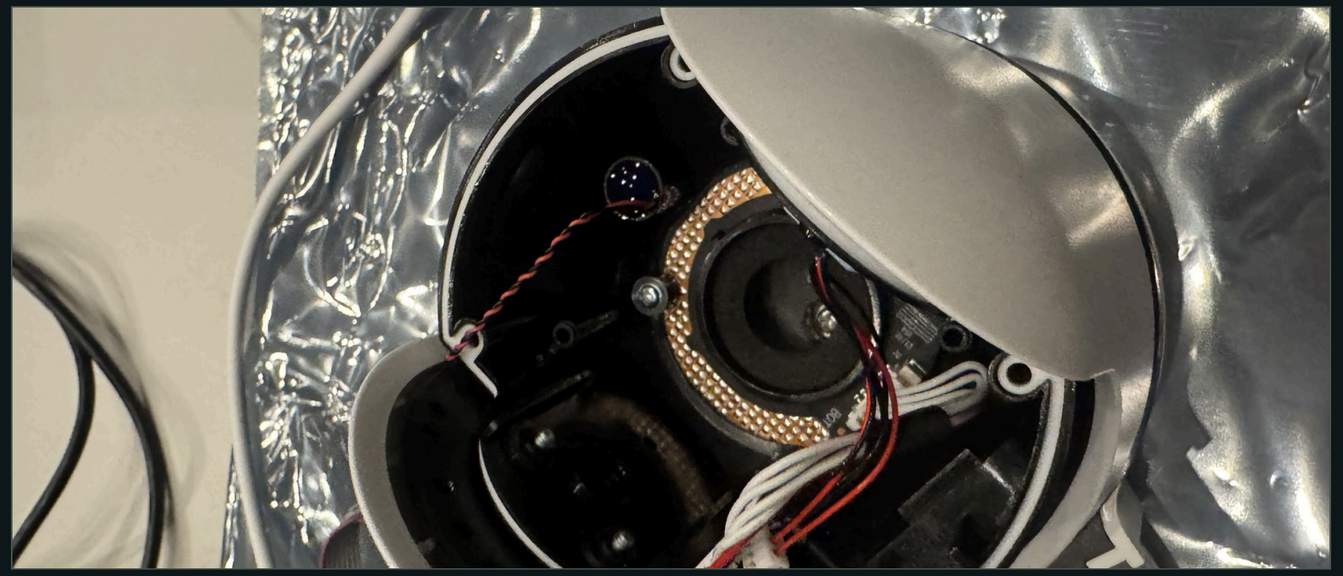
OBSERVER → RETROUVER ET VÉRIFIER LES CLÉS → SIGNER LES 22 ORDRES

L'IDÉE À RETENIR

Le bug du nonce et les messages prévisibles ont permis **de retrouver les clés de signature.**

LE DÉFI · DU MATÉRIEL BIEN RÉEL

Elle surveillait le jardin.
On a regardé à l’intérieur.



TAPO C500 2.0

Une caméra.
Un petit ordinateur.

La mémoire conserve le logiciel,
les réglages et des fichiers.

NOTRE OBJECTIF

BA26{…}

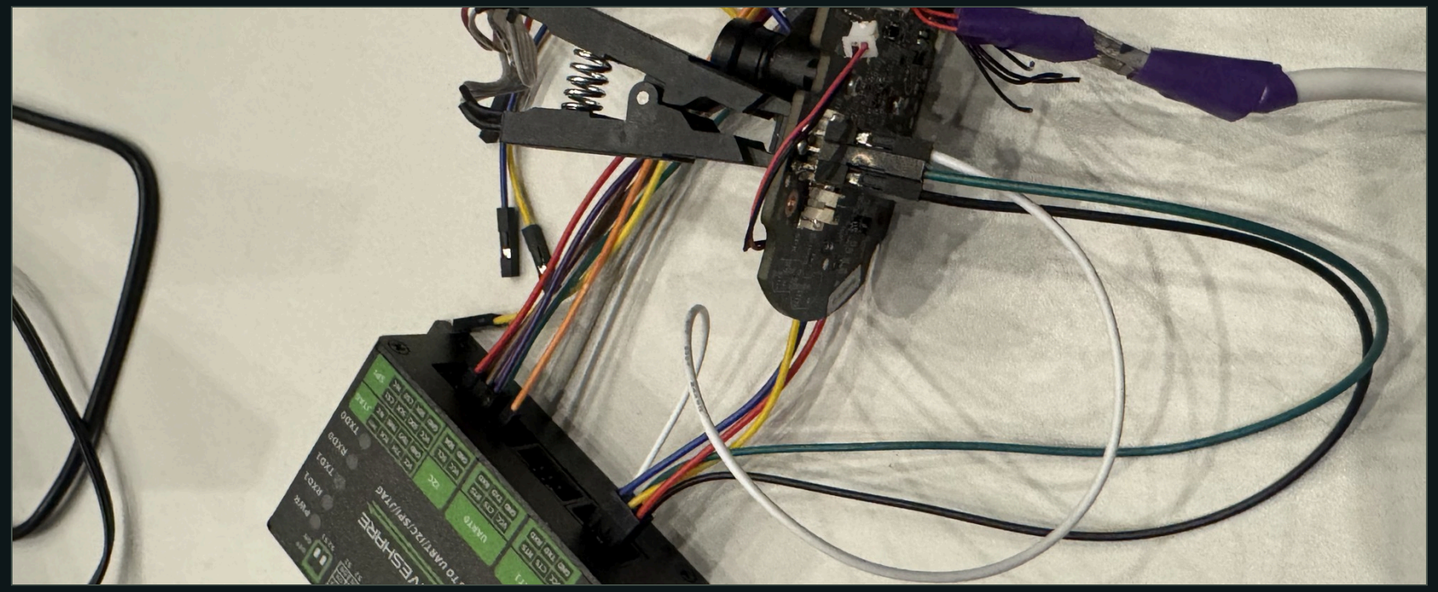
Retrouver le message caché.

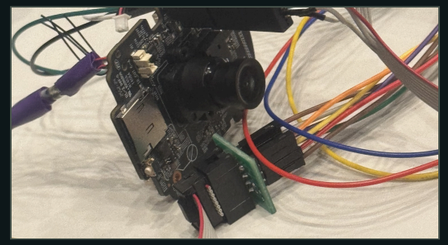
L’IDÉE À RETENIR

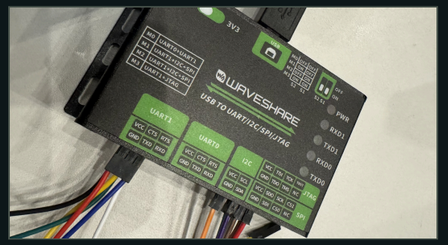
Le point de départ de l’analyse : **une copie de la mémoire de 8 Mio.**

L’ACQUISITION · COPIER LA MÉMOIRE POUR L’ÉTUDIER

Beaucoup de fils.
À l’arrivée : un fichier.







Mémoire flash

↓ lecture

Adaptateur → ordinateur

↓ copie

dump · 8 Mio

L’IDÉE À RETENIR

On poursuit l’enquête **dans le fichier récupéré.**

PREMIER ESSAI · LA RECHERCHE DE TEXTE NE SUFFIT PAS

Ctrl+F n’a rien trouvé.
Il fallait changer d’échelle.

01 / STOCKAGE

Des fichiers compressés

Prendre moins de place.



→ `fichiers`

Décompresser pour accéder au contenu.

02 / MESSAGE

Un flag chiffré

Rendre le texte illisible.

`9c b6 c8 70 d8 3a…`

Comprendre la recette pour le retrouver.

L’IDÉE À RETENIR

Ce défi combine **compression des fichiers** et **chiffrement du message.**

LE DÉCLIC · UNE ZONE UTILE AVAIT ÉCHAPPÉ À L’EXTRACTION

Le livre avait perdu sa couverture. Pas ses pages.

DUMP · SCHÉMA NON À L’ÉCHELLE



DÉBUT NON RECONNAISSABLE

?

La première extraction ne voit pas tout.

→

MÉTADONNÉES INTACTES

Position	Où lire
Taille	Combien lire
Fragment	Quel bloc ouvrir

→

FICHIERS RÉCUPÉRÉS

`ctf_unlock`

`flag.enc`

Lire les bons octets.

L’IDÉE À RETENIR

L’outil n’avait pas tout vu. La mémoire avait encore des choses à raconter.

LES INDICES · TROIS PIÈCES QUI SE COMPLÈTENT

Le message était secret.
Le mode d’emploi était à côté.

01

flag.enc

Le message chiffré.
35 octets dans le fichier fourni.

+

02

ctf_unlock

Le programme qui explique comment retrouver le texte.

+

03

devId

L’identifiant de la caméra, présent dans ses réglages.

AU DÉMARRAGE

```
devId → clé temporaire → ctf_unlock + flag.enc → /tmp/flag.txt
```

L’IDÉE À RETENIR

Le démarrage relie les indices : **identifiant + programme + message chiffré.**

LE MÉCANISME · REPRODUIRE LA RECETTE

Mêmes ingrédients.
Exactement le même masque.

`devId · identifiant` + `« C500-CTF-v1 »` + `compteur : 0, puis 1`

↓ `SHA-256` Une empreinte de 32 octets par compteur.

`Bloc 0 · 32 octets` `Bloc 1 · 32 octets`

Le message fait **35 octets** : on utilise le premier bloc, puis 3 octets du second.

L'IDÉE À RETENIR

On calcule SHA-256 dans le sens habituel. **On ne l'inverse pas.**

PETITE PAUSE · LE PUBLIC PEUT AUSSI DÉCHIFFRER

Une opération qui sait faire marche arrière.

XOR

Deux bits identiques → **0**

Deux bits différents → **1**

Exemple pédagogique de 4 bits.
Ce ne sont pas les données du flag.

Message

1 0 1 0

Masque

0 1 1 0

Chiffré

1 1 0 0

Chiffré

1 1 0 0

Même masque

0 1 1 0

Message

1 0 1 0

L'IDÉE À RETENIR

Appliquer **le même masque deux fois** rend le message d'origine.

DU SCHÉMA AU CODE · UN EXTRAIT, TROIS GESTES

La recette tient en Python. Chaque ligne a un rôle.

01	`key + b'C500-CTF-v1' + struct.pack('<I', counter)`	Assembler les ingrédients. Le compteur prend 0, puis 1.
02	`hashlib.sha256(…).digest()`	Produire un bloc de 32 octets. Mettre les deux blocs bout à bout.
03	`bytes(a ^ b for a, b in zip(encrypted, stream))`	Combiner chaque octet chiffré avec l'octet du masque.

Extraits de solve.py, séparés pour la lecture ; « … » abrège les ingrédients. Le script n'exécute pas ctf_unlock.

L'IDÉE À RETENIR

Le script relie les fichiers extraits, l'identifiant et la recette. **Le résultat devient vérifiable.**

LE RÉSULTAT · TEXTE CONSERVÉ DANS FLAG.TXT

Souriez.
Le flag est sorti.

CONTENU EXACT DU FICHIER FLAG.TXT FOURNI

```
BA26{a-R00t-0f-Tru5t-Sh1ps-N0-K3ys}
```

35 caractères · même résultat que dans le PowerPoint.

ÉTAT DOCUMENTÉ **Flag validé.** Soumission confirmée par la plateforme.

L'IDÉE À RETENIR

Le secret partageait sa mémoire avec **sa recette et les données nécessaires pour le lire.**

DU RÉSULTAT AU RAISONNEMENT

Le flag termine la recherche.
La preuve raconte le solve.

01	Identifier le verrou	Qu’est-ce que le système vérifie vraiment ?
02	Trouver l’hypothèse fragile	Un identifiant, un score ou des nombres suffisent-ils ?
03	Confirmer le résultat	Une sortie conservée, un test de clé, un flag réel.

L’IDÉE À RETENIR

Les agents ont trouvé les chemins. **Nous pouvons maintenant les expliquer.**